

Textové funkce v PostgreSQL na databázi Sakila

Příklady používají schema `public` a plně kvalifikované názvy tabulek, takže není nutné předem nastavovat `search_path`.

Všechny příklady jsou pouze dotazy `SELECT` a nemění data.

1. Změna velikosti písmen

Funkce `UPPER` převede text na velká písmena, `LOWER` na malá písmena a `INITCAP` nastaví první písmeno každého slova jako velké.

```
SELECT
    actor_id,
    first_name,
    last_name,
    UPPER(first_name) AS first_name_upper,
    LOWER(last_name) AS last_name_lower,
    INITCAP(first_name || ' ' || last_name) AS full_name_pretty
FROM public.actor
ORDER BY actor_id
LIMIT 10;
```

Operátor `||` spojuje textové hodnoty. Pokud by některá hodnota byla `NULL`, výsledkem celého spojení by také bylo `NULL`; pro bezpečné spojování je vhodnější `CONCAT_WS`, který je uveden níže.

2. Spojení jména a e-mailu

`CONCAT_WS` spojí hodnoty pomocí zadaného oddělovače. Hodnoty `NULL` při spojování přeskočí.

```
SELECT
    customer_id,
    CONCAT_WS(' ', first_name, last_name) AS full_name,
    CONCAT_WS(' | ', CONCAT_WS(' ', first_name, last_name), email) AS
customer_contact
FROM public.customer
ORDER BY last_name, first_name
LIMIT 20;
```

3. Délka textu

`LENGTH` vrací počet znaků v textové hodnotě. Dotaz vyhledá filmy s delším názvem.

```
SELECT
    film_id,
    title,
```

```
LENGTH(title) AS title_length
FROM public.film
WHERE LENGTH(title) >= 20
ORDER BY title_length DESC, title;
```

4. Začátek a konec textu

LEFT vrátí zadaný počet znaků zleva a **RIGHT** stejným způsobem zprava.

```
SELECT
    film_id,
    title,
    LEFT(title, 8) AS title_beginning,
    RIGHT(title, 8) AS title_ending
FROM public.film
ORDER BY film_id
LIMIT 15;
```

5. Vyhledání části textu

POSITION vrací pozici prvního výskytu hledaného textu. Pokud se text nenajde, vrací hodnotu 0.

```
SELECT
    film_id,
    title,
    POSITION('A' IN UPPER(title)) AS first_a_position
FROM public.film
WHERE POSITION('A' IN UPPER(title)) > 0
ORDER BY first_a_position, title;
```

Pro jednodušší filtrování lze použít **LIKE** nebo **ILIKE**. **ILIKE** nerozlišuje velká a malá písmena.

```
SELECT
    film_id,
    title
FROM public.film
WHERE title ILIKE '%love%'
ORDER BY title;
```

6. Výběr části textu pomocí **SUBSTRING**

SUBSTRING vybere část řetězce od zadané pozice a délky.

```
SELECT
    customer_id,
```

```
email,  
SUBSTRING(email FROM 1 FOR POSITION('@' IN email) - 1) AS email_user,  
SUBSTRING(email FROM POSITION('@' IN email) + 1) AS email_domain  
FROM public.customer  
WHERE email IS NOT NULL  
ORDER BY email;
```

Podmínka `email IS NOT NULL` je důležitá, protože textové funkce vracejí při vstupu `NULL` obvykle také `NULL`.

7. Rozdělení textu pomocí `SPLIT_PART`

`SPLIT_PART` rozdělí text podle oddělovače a vrátí část na zadaném pořadí. V tomto příkladu získáme doménu e-mailu za znakem `@`.

```
SELECT  
    customer_id,  
    email,  
    SPLIT_PART(email, '@', 2) AS email_domain  
FROM public.customer  
WHERE email IS NOT NULL  
ORDER BY email_domain, email;
```

8. Nahrazení části textu

`REPLACE` nahradí všechny výskyty hledaného podřetězce.

```
SELECT  
    film_id,  
    title,  
    REPLACE(title, ' ', '_') AS title_for_url  
FROM public.film  
ORDER BY film_id  
LIMIT 20;
```

Tento příklad nahrazuje mezery podtržítkem. Jde pouze o výpočet ve výsledku dotazu; hodnota ve sloupci `title` zůstává beze změny.

9. Odstranění mezer a práce s `NULL`

`TRIM` ve výchozím nastavení odstraní mezery na začátku a konci textu, nikoli například tabulátory. `COALESCE` vrátí první hodnotu, která není `NULL`.

```
SELECT  
    address_id,  
    TRIM(address) AS cleaned_address,  
    COALESCE(NULLIF(TRIM(address2), ''), 'bez další adresy') AS address_line_2  
FROM public.address
```

```
ORDER BY address_id
LIMIT 20;
```

`NULLIF(TRIM(address2), '')` nejprve převede prázdný text na `NULL`. `COALESCE` pak pro `NULL` použije náhradní text.

10. Zarovnání textu pomocí `LPAD` a `RPAD`

PostgreSQL používá pro doplnění textu funkce `LPAD` a `RPAD`. `LPAD` doplňuje znaky zleva, `RPAD` zprava. Druhý argument určuje výslednou délku a třetí argument znak nebo text, kterým se doplňuje.

```
SELECT
    film_id,
    title,
    LPAD(film_id::text, 5, '0') AS film_code,
    RPAD(LEFT(title, 20), 20, '.') AS title_fixed_width
FROM public.film
ORDER BY film_id
LIMIT 20;
```

Například identifikátor `7` bude pomocí `LPAD` zobrazen jako `00007`. `RPAD` doplní zkrácený název filmu tečkami do délky 20 znaků. Pokud je původní text delší než požadovaná délka, PostgreSQL jej ořízne.

```
SELECT
    LPAD('7', 5, '0') AS left_padding,
    RPAD('Sakila', 10, '-') AS right_padding;
```

Výsledkem bude `00007` a `Sakila-----`. Padding mění pouze hodnotu ve výsledku dotazu, nikoli uložená data.

11. Textové funkce společně s `CASE`

`CASE` umožňuje vytvořit textový popis podle délky názvu filmu.

```
SELECT
    film_id,
    title,
    LENGTH(title) AS title_length,
    CASE
        WHEN LENGTH(title) < 10 THEN 'krátký název'
        WHEN LENGTH(title) < 20 THEN 'středně dlouhý název'
        ELSE 'dlouhý název'
    END AS title_category
FROM public.film
ORDER BY title_length, title;
```

Přehled nejčastějších funkcí

Funkce	Účel
UPPER, LOWER, INITCAP	změna velikosti písmen
CONCAT, CONCAT_WS,	spojování textu
LENGTH	délka textu
LEFT, RIGHT, SUBSTRING	výběr části textu
POSITION	nalezení pozice podřetězce
LIKE, ILIKE	vyhledání podle vzoru
SPLIT_PART	rozdělení textu podle oddělovače
REPLACE	nahrazení podřetězce
TRIM	odstranění okrajových mezer
COALESCE, NULLIF	ošetření hodnot NULL a prázdných textů
LPAD, RPAD	doplnění textu zleva nebo zprava

Při tvorbě tohoto dokumentu byl využit GitHub Copilot.